Recursion and Backtracking in Java

Core ideas, execution flow, practical examples, and common mistakes

Purpose of these notes
These notes explain how recursive methods work, how to trace them, and how backtracking uses recursion to explore possible choices. The examples use Java and are written to be compiled and tested independently.

1. Recursion: the basic idea

Recursion is a technique in which a method solves a problem by calling itself with a smaller or simpler input. Instead of solving the entire problem in one step, it reduces the problem until it reaches a case whose answer is already known.

A simple real-life analogy is opening nested boxes: each box contains a smaller box, until the final box is empty. The process must eventually stop, and then the results are handled while returning through the opened boxes.

The two essential parts

Base case: the condition that stops further recursive calls. Without a valid base case, the method may keep calling itself until Java throws a `StackOverflowError`.

Recursive case: the part that makes progress toward the base case, usually by reducing the input or moving to the next position.

```
static void countDown(int n) {
    if (n == 0) {                 // base case
        System.out.println("Done");
        return;
    }
    System.out.println(n);
    countDown(n - 1);             // smaller problem
}
```

Calling `countDown(3)` prints 3, 2, 1, and then Done. Each call has its own value of `n`. The calls do not overwrite one another; Java keeps their execution state on the call stack.

A quick checklist when writing recursion

1. What is the smallest input I can answer directly? 2. How does each call make the problem smaller? 3. Does every valid input eventually reach the base case?

2. How recursive calls execute

A recursive method does not finish immediately when it calls itself. The current call pauses at the call site, the next call runs, and so on. Once a base case returns, the paused calls resume in reverse order. This is the last-in, first-out behaviour of the call stack.

Example: factorial

For a non-negative integer n, factorial is n × (n – 1) × ... × 1, and 0! is defined as 1. The recursive definition is factorial(n) = n × factorial(n – 1), with factorial(0) = 1.

```
static long factorial(int n) {
    if (n < 0) {
        throw new IllegalArgumentException("n must be non-negative");
    }
    if (n == 0 || n == 1) {
        return 1;
    }
    return n * factorial(n - 1);
}
```

Trace factorial(4)

Phase	What happens
Going down	factorial(4) waits for factorial(3)
Going down	factorial(3) waits for factorial(2)
Going down	factorial(2) waits for factorial(1)
Base case	factorial(1) returns 1
Returning	factorial(2) = 2 × 1 = 2
Returning	factorial(3) = 3 × 2 = 6
Returning	factorial(4) = 4 × 6 = 24

The important point is that multiplication happens during the return phase. The base case gives the first known answer, and each waiting call uses it to calculate its own result.

Recursion versus iteration

A loop repeats a block in one method frame; recursion creates method calls. Both can solve many of the same problems. Loops usually use less call-stack memory, while recursion can express divide-and-conquer, tree traversal, and search problems more naturally.

Complexity note: factorial performs O(n) calls and uses O(n) call-stack space. Recursive code is not automatically faster or more memory-efficient than a loop.

3. Common recursion patterns

A. Sum from 1 to n

The sum up to n can be written as n + sum(n – 1). The base case is n = 0, whose sum is 0.

```
static int sumToN(int n) {
    if (n <= 0) return 0;
    return n + sumToN(n - 1);
}
// sumToN(4) -> 4 + 3 + 2 + 1 + 0 = 10
```

B. Print a string in reverse

The recursive call can move to the next character first, then print the current character while calls return. This demonstrates why code placed before or after a recursive call changes the output order.

```
static void printReverse(String s, int i) {
    if (i == s.length()) return;
    printReverse(s, i + 1);
    System.out.print(s.charAt(i));
}
// printReverse("JAVA", 0) prints AVAJ
```

C. Fibonacci numbers

Fibonacci numbers begin 0, 1, 1, 2, 3, 5, 8... Each term after the first two is the sum of the previous two. The direct recursive version mirrors the definition, but repeats work.

```
static int fib(int n) {
    if (n <= 1) return n;
    return fib(n - 1) + fib(n - 2);
}
```

This simple Fibonacci implementation takes exponential time, commonly expressed as O(2^n) as an upper-bound estimate, because many subproblems are solved repeatedly. Memoization or an iterative solution reduces the work to O(n).

D. Recursion on arrays

An index can represent progress through an array. For example, the following method checks whether a target exists from the current index onward.

```
static boolean contains(int[] a, int i, int target) {
    if (i == a.length) return false;
    if (a[i] == target) return true;
    return contains(a, i + 1, target);
}
```

How to trace any recursive method

Write each call on a separate line with its argument. Mark where it reaches the base case. Then trace the return values upward. If the method prints, also mark whether each print statement is before or after the recursive call.

Common mistakes: forgetting the base case, changing the argument in the wrong direction, returning the wrong value, or assuming statements after the recursive call run before the deeper calls.

4. Backtracking: explore, test, undo

Backtracking is a problem-solving approach that builds a candidate solution one decision at a time. If a choice leads to a state that cannot produce a valid answer, the algorithm reverses that choice and tries another option.

A typical backtracking algorithm has three steps: **choose** an option, **explore** the resulting state recursively, and **undo** the choice before trying the next option. Recursion provides the exploration structure; backtracking adds the systematic undo-and-try-again process.

A simple example: generate all binary strings of length n

At every position, there are two choices: place 0 or place 1. Once the string reaches length n, print it. After exploring the 0 branch, change that position to 1 and explore the other branch.

```
static void generate(String built, int n) {
    if (built.length() == n) {
        System.out.println(built);
        return;
    }

    generate(built + "0", n);
    generate(built + "1", n);
}
// generate("", 2) prints: 00, 01, 10, 11
```

This version creates new strings, so it does not need to manually undo a character. In many backtracking problems, a mutable list or array is used instead; then the algorithm must remove or reset the choice after the recursive call.

Backtracking template

```
void search(State state) {
    if (isComplete(state)) {
        recordAnswer(state);
        return;
    }

    for (Choice choice : availableChoices(state)) {
        if (!isValid(choice, state)) continue;

        apply(choice, state);       // choose
        search(state);              // explore
        undo(choice, state);        // backtrack
    }
}
```

The exact state and validity test depend on the problem. The undo step must restore the state to what it was before the choice was made; otherwise later branches may accidentally inherit earlier choices.

Backtracking and pruning

Pruning means rejecting a partial candidate as soon as it is clear that it cannot lead to a valid answer. For example, when solving a maze, there is no reason to continue through a wall or revisit a cell already on the current path if the rules forbid it. Good pruning can dramatically reduce the search space.

5. Backtracking example: generate subsets

A subset problem asks us to consider which elements to include or exclude. For each array element, the algorithm explores two branches: include it in the current subset, or skip it.

```
import java.util.ArrayList;
import java.util.List;

public class Subsets {
    static void generate(int[] nums, int index,
                         List<Integer> current) {
        if (index == nums.length) {
            System.out.println(current);
            return;
        }

        // Choice 1: include nums[index]
        current.add(nums[index]);
        generate(nums, index + 1, current);

        // Undo the include choice
        current.remove(current.size() - 1);

        // Choice 2: exclude nums[index]
        generate(nums, index + 1, current);
    }

    public static void main(String[] args) {
        int[] nums = {1, 2, 3};
        generate(nums, 0, new ArrayList<>());
    }
}
```

How the choices branch

For the input [1, 2], the possible subsets are [], [1], [2], and [1, 2]. Each element creates an include/exclude decision, so n elements produce 2^n subsets. For n = 3, there are 8 subsets.

Notice the call to remove the last element. It reverses the earlier add operation. Without it, the branch that excludes an element could still contain that element from the include branch, causing incorrect results.

Where backtracking is used

Maze solving: try a valid direction, explore the path, and return when blocked.

N-Queens: place queens row by row while checking that no two queens attack each other.

Sudoku: place a valid digit in an empty cell, solve the remaining board, and undo the digit if it causes a contradiction.

Permutations and combinations: build ordered arrangements or selections while tracking which elements have already been used.

Word search: explore adjacent cells while marking the current path and unmarking cells when returning.

Complexity

Backtracking complexity depends on the number of candidates and the cost of checking each one. Subset generation produces 2^n answers and therefore needs at least O(2^n) output work, ignoring the cost of printing each subset. Other problems, such as permutations, can have factorial-sized search spaces.

6. Recursion vs. backtracking + revision

Aspect	Recursion	Backtracking
Main idea	A method calls itself on a smaller or related problem.	Explore choices and undo them when returning.
Base case	Defines when a call stops.	Often identifies a complete candidate or a stopping condition.
Undo step	Not required in every recursive method.	Usually essential when shared mutable state is changed.
Typical uses	Factorial, tree traversal, divide and conquer.	Subsets, permutations, maze, N-Queens, Sudoku.
Search behaviour	May follow one chain or split into multiple calls.	Systematically explores alternative choices.

A practical checklist for debugging

• Verify the base case with the smallest input.
• Check that every recursive call moves toward termination.
• Trace both the descent and return phases.
• For backtracking, confirm that every applied choice is undone on every relevant return path.
• Test an empty input, a one-element input, and a case with no valid solution where appropriate.

Practice questions

1. Write a recursive method to calculate the sum of digits of a positive integer.
2. Write a recursive method to check whether a string is a palindrome.
3. Generate all binary strings of length 3 without consecutive 1s.
4. Generate all permutations of an array of distinct integers.
5. Explain why a subset-generation algorithm produces 2^n results.
6. In the subset code, what bug appears if current.remove(...) is deleted?

Key takeaways

Recursion breaks a problem into smaller calls and relies on a base case to stop. Backtracking uses recursive exploration to try alternatives, usually changing state before a call and restoring it afterward. The most reliable way to understand both is to trace the calls, the choices, and the return path on paper before running the code.

> Suggested next step: compile each code example, trace one small input by hand, and write the practice solutions without looking at the notes.